

Name: Irtiza Ali

Internship Domain: Data Science

Task Number: 04

Technologies Used: Python, Pandas, Power BI and Google Colab.

Dataset Description:

The Supermart Grocery Sales dataset contains **9,994 sales records** with information about customers, product categories, sub-categories, cities, regions, order dates, sales, discounts, and profit. The dataset covers the period from **2015 to 2018** and is suitable for analyzing sales performance, customer behavior, product performance, and profitability.

Dataset Source: https://www.kaggle.com/datasets/mohamedharris/supermart-grocery-sales-retail-analytics-dataset?utm_source=chatgpt.com

GitHub Repository Link:

<https://github.com/irtizaali283/Supermart-Grocery-Sales-Analytics-and-Business-Intelligence-Dashboard.git>

Business Insights:

1. Total sales are 14,956,982 and total profit is 3,747,121.20, with an overall profit margin of approximately 25.05%.
2. Sales increased every year, rising from 2,975,599 in 2015 to 4,977,512 in 2018.
3. In 2018, sales increased by approximately 28.55% compared with 2017, while profit increased by around 30.52%.
4. Q4 is the strongest quarter, generating sales of 5,525,289 and profit of 1,395,461.39.
5. November is the strongest month, with sales of 2,193,924, while February has the lowest sales at 456,102.
6. Eggs, Meat & Fish is the highest-selling category, generating sales of 2,267,401.
7. Snacks generates the highest category profit at 568,178.85 and also has the highest category profit margin at approximately 25.39%.
8. Oil & Masala has the lowest category sales and profit, along with the lowest category profit margin of approximately 24.43%.
9. Health Drinks is the highest-selling sub-category, with sales of 1,051,439.
10. Noodles has the highest sub-category profit margin at approximately 26.34%.
11. Spices has the lowest sub-category profit margin at approximately 23.82%.
12. The West region has the highest sales at 4,798,743 and the highest profit at 1,192,004.61.
13. The North region contains only one record, so its performance should not be directly compared with the other regions.

14. Kanyakumari is the highest-sales city, with sales of 706,764, while Vellore generates the highest city profit at 174,073.01.
15. Krithika is the highest-sales customer, with sales of 334,361, while Arutra generates the highest customer profit at 87,572.40.

Bonus Forecast Insight:

The sales forecasting model achieved an average percentage error of approximately 10.8% on the test data and was used to forecast sales for the next six months.

Business Recommendation:

1. Maintain strong inventory availability for Eggs, Meat & Fish because it generates the highest category sales.
2. Give Snacks strong shelf placement and marketing support because it combines high sales with the highest category profit.
3. Expand or promote Health Drinks because it is the highest-selling sub-category.
4. Review pricing, product mix, and costs in Oil & Masala because it has the weakest category sales, profit, and profit margin.
5. Protect and expand Noodles where possible because it has the highest sub-category profit margin.
6. Review Spices costs and pricing because it has the lowest sub-category profit margin.
7. Prepare additional inventory, staffing, and promotions for Q4, especially November and December, because demand is highest during this period.
8. Use the West region as a key revenue market while studying its successful product mix for possible use in other regions.
9. Investigate the single North region record before making regional decisions because it may represent a classification or coverage issue.
10. Avoid increasing discounts as a general profit strategy. The dataset shows almost no direct relationship between discount and profit, so discount campaigns should be tested by category or product group.

Bonus Recommendation:

Use the sales forecast as an additional input for inventory and staffing planning, while regularly updating the model when new sales data becomes available.

Model Used:

Holt-Winters Exponential Smoothing Model

Preprocessing:

Imports the required libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Loads the dataset into a Pandas DataFrame.

```
df = pd.read_csv("Supermart Grocery Sales - Retail Analytics Dataset.csv")
```

Checking Rows & Cols

```
df.shape
```

```
(9994, 11)
```

Displaying the starting 5 rows for better understanding of structure

```
df.head()
```

	Order ID	Customer Name	Category	Sub Category	City	Order Date	Region	Sales	Discount	Profit	State
0	OD1	Harish	Oil & Masala	Masalas	Vellore	11-08-2017	North	1254	0.12	401.28	Tamil Nadu
1	OD2	Sudha	Beverages	Health Drinks	Krishnagiri	11-08-2017	South	749	0.18	149.80	Tamil Nadu
2	OD3	Hussain	Food Grains	Atta & Flour	Perambalur	06-12-2017	West	2360	0.21	165.20	Tamil Nadu
3	OD4	Jackson	Fruits & Veggies	Fresh Vegetables	Dharmapuri	10-11-2016	South	896	0.25	89.60	Tamil Nadu
4	OD5	Ridhesh	Food Grains	Organic Staples	Ooty	10-11-2016	South	2355	0.26	918.45	Tamil Nadu

Displays all column names in the dataset.

```
df.columns
```

```
Index(['Order ID', 'Customer Name', 'Category', 'Sub Category', 'City',  
      'Order Date', 'Region', 'Sales', 'Discount', 'Profit', 'State'],  
      dtype='object')
```

Shows the data type of each column.

```
df.dtypes
```

```
Order ID      object
Customer Name  object
Category       object
Sub Category   object
City           object
Order Date     object
Region         object
Sales          int64
Discount       float64
Profit         float64
State          object
dtype: object
```

Checks how many missing values are present in each column.

```
( ) df.isnull().sum()
v Order ID      0
  Customer Name 0
  Category      0
  Sub Category  0
  City          0
  Order Date    0
  Region        0
  Sales         0
  Discount      0
  Profit        0
  State         0
  dtype: int64
```

Checks whether any completely duplicated records exist.

```
( ) df.duplicated().sum()
v np.int64(0)
```

Shows the number of unique values in each column.

```
( ) df.nunique()
v Order ID      9994
  Customer Name 50
  Category      7
  Sub Category  23
  City          24
  Order Date    1236
  Region        5
  Sales         1989
  Discount      26
  Profit        8380
  State         1
  dtype: int64
```

Displays summary statistics for the numerical columns.

```
( ) df.describe()
v
```

	Sales	Discount	Profit
count	9994.000000	9994.000000	9994.000000
mean	1406.596158	0.228817	374.937082
std	577.559036	0.074636	239.932881
min	500.000000	0.100000	25.250000
25%	1000.000000	0.180000	180.022500
50%	1498.000000	0.230000	320.780000
75%	1994.750000	0.290000	525.627500
max	2500.000000	0.350000	1120.950000

Displays summary statistics for the text columns.

```
df.describe(include="object")
```

	Order ID	Customer Name	Category	Sub Category	City	Order Date	Region	State
count	9994	9994	9994	9994	9994	9994	9994	9994
unique	9994	50	7	23	24	1238	5	1
top	OD1	Amrith	Snacks	Health Drinks	Kanyakumari	09-05-2017	West	Tamil Nadu
freq	1	227	1514	719	459	38	3203	9994

Checks whether every Order ID is unique.

```
df["Order ID"].nunique()
```

```
9994
```

Shows how many records belong to each region.

```
df["Region"].value_counts()
```

```
Region
West      3203
East      2848
Central   2323
South     1619
North       1
Name: count, dtype: int64
```

Shows how many records belong to each state.

```
df["State"].value_counts()
```

```
State
Tamil Nadu    9994
Name: count, dtype: int64
```

Shows how many records belong to each product category.

```
df["Category"].value_counts()
```

```
Category
Snacks      1514
Eggs, Meat & Fish  1490
Fruits & Veggies  1418
Bakery      1413
Beverages   1400
Food Grains 1398
Oil & Masala 1361
Name: count, dtype: int64
```

Shows how many records belong to each product sub-category.

```
df["Sub Category"].value_counts()
```

```
Sub Category
Health Drinks    719
Soft Drinks     681
Cookies         520
Breads & Buns    502
Chocolates      499
```

Displays sample order dates to inspect their format.

```
df["Order Date"].head(20)
```

```
0    11-08-2017
1    11-08-2017
2    06-12-2017
3    10-11-2016
4    10-11-2016
5    06-09-2015
6    06-09-2015
7    06-09-2015
8    06-09-2015
9    06-09-2015
10   06-09-2015
11   06-09-2015
12    4/15/2018
13   12-05-2017
14   11/22/2016
15   11/22/2016
16   11-11-2015
17    5/13/2015
18    8/27/2015
19    8/27/2015
Name: Order Date, dtype: object
```

Converts Order Date from text into a proper datetime data type.

```
df["Order Date"] = pd.to_datetime(df["Order Date"], format="mixed")
```

Confirms that Order Date was converted successfully.

```
df["Order Date"].dtype
```

```
dtype('datetime64[ns]')
```

Checks the earliest and latest dates available in the dataset.

```
print("Earliest Date:", df["Order Date"].min())
print("Latest Date:", df["Order Date"].max())
```

```
--- Earliest Date: 2015-01-03 00:00:00
    Latest Date: 2018-12-30 00:00:00
```

Checks whether Sales contains zero or negative values.

```
df[df["Sales"] <= 0]
```

Order ID	Customer Name	Category	Sub Category	City	Order Date	Region	Sales	Discount	Profit	State
----------	---------------	----------	--------------	------	------------	--------	-------	----------	--------	-------

Checks the minimum and maximum discount values.

```
print("Minimum Discount:", df["Discount"].min())
print("Maximum Discount:", df["Discount"].max())
```

```
Minimum Discount: 0.1
Maximum Discount: 0.35
```

Checks whether any orders have zero or negative profit.

```
(df["Profit"] <= 0).sum()  
  
np.int64(0)
```

Outlier Analysis

Calculates the IQR limits for Sales.

```
sales_q1 = df["Sales"].quantile(0.25)  
sales_q3 = df["Sales"].quantile(0.75)  
sales_iqr = sales_q3 - sales_q1  
sales_lower = sales_q1 - (1.5 * sales_iqr)  
sales_upper = sales_q3 + (1.5 * sales_iqr)  
  
print("Lower Limit:", sales_lower)  
print("Upper Limit:", sales_upper)  
  
... Lower Limit: -492.125  
Upper Limit: 3486.875
```

Counts Sales values that fall outside the IQR limits.

```
((df["Sales"] < sales_lower) | (df["Sales"] > sales_upper)).sum()  
  
np.int64(0)
```

Calculates the IQR limits for Profit.

```
profit_q1 = df["Profit"].quantile(0.25)  
profit_q3 = df["Profit"].quantile(0.75)  
profit_iqr = profit_q3 - profit_q1  
profit_lower = profit_q1 - (1.5 * profit_iqr)  
profit_upper = profit_q3 + (1.5 * profit_iqr)  
  
print("Lower Limit:", profit_lower)  
print("Upper Limit:", profit_upper)  
  
Lower Limit: -338.385  
Upper Limit: 1044.035
```

Counts Profit values that fall outside the IQR limits.

```
((df["Profit"] < profit_lower) | (df["Profit"] > profit_upper)).sum()  
  
np.int64(43)
```

Displays the highest Profit outliers for inspection.

```
df[(df["Profit"] < profit_lower) |  
   (df["Profit"] > profit_upper)  
].sort_values("Profit", ascending=False).head(10)
```

	Order_ID	Customer Name	Category	Sub Category	City	Order Date	Region	Sales	Discount	Profit	State
3159	OD3180	Haseena	Bakery	Cakes	Cumbum	2017-09-04	Central	2491	0.26	1120.95	Tamil Nadu
3467	OD3488	Verma	Fruits & Veggies	Fresh Fruits	Theni	2015-02-18	Central	2490	0.24	1120.50	Tamil Nadu
3436	OD3437	Yadav	Bakery	Breads & Buns	Theni	2015-12-18	Central	2489	0.29	1111.05	Tamil Nadu
8134	OD8135	Aditi	Bakery	Biscuits	Coimbatore	2016-09-18	East	2452	0.18	1103.40	Tamil Nadu
9782	OD9783	Komal	Snacks	Cookies	Karur	2017-03-29	Central	2450	0.21	1102.50	Tamil Nadu
1115	OD1116	Vinne	Eggs, Meat & Fish	Chicken	Madurai	2017-07-04	West	2439	0.30	1097.55	Tamil Nadu
4943	OD4944	Williams	Beverages	Soft Drinks	Ramanadhapuram	2018-11-21	Central	2434	0.29	1095.30	Tamil Nadu
1304	OD1305	Akash	Fruits & Veggies	Organic Vegetables	Salem	2017-12-01	East	2432	0.34	1094.40	Tamil Nadu
1807	OD1808	Esther	Bakery	Biscuits	Trichy	2018-11-21	West	2429	0.11	1093.05	Tamil Nadu
3144	OD3145	Sabeela	Oil & Masala	Masalas	Krishnagiri	2017-12-29	Central	2478	0.13	1090.32	Tamil Nadu

The Profit outliers are kept because they are valid high-profit transactions and do not show missing or invalid values.

Feature Engineering

Creates an Order Year column for yearly analysis.

```
df["Order Year"] = df["Order Date"].dt.year
```

Creates an Order Month column for monthly analysis.

```
df["Order Month"] = df["Order Date"].dt.month_name()
```

Creates a Month Number column so months can be sorted correctly.

```
df["Month Number"] = df["Order Date"].dt.month
```

Creates an Order Quarter column for quarterly analysis.

```
df["Order Quarter"] = "Q" + df["Order Date"].dt.quarter.astype(str)
```

Calculates the profit margin percentage for each order.

```
df["Profit Margin"] = (df["Profit"] / df["Sales"]) * 100
```

Calculates the profit margin percentage for each order.

```
( ) df["Profit Margin"] = (df["Profit"] / df["Sales"]) * 100
```

Displays the dataset after feature engineering.

```
( ) df.head()
```

	Order ID	Customer Name	Category	Sub Category	City	Order Date	Region	Sales	Discount	Profit	State	Order Year	Order Month	Month Number	Order Quarter	Profit
0	OD1	Harish	Oil & Masala	Masalas	Vellore	2017-11-08	North	1254	0.12	401.28	Tamil Nadu	2017	November	11	Q4	
1	OD2	Sudha	Beverages	Health Drinks	Krishnagiri	2017-11-08	South	749	0.18	149.80	Tamil Nadu	2017	November	11	Q4	
2	OD3	Hussain	Food Grains	Atta & Flour	Perambalur	2017-08-12	West	2380	0.21	185.20	Tamil Nadu	2017	June	6	Q2	
3	OD4	Jackson	Fruits & Veggies	Fresh Vegetables	Dharmapuri	2018-10-11	South	896	0.25	89.60	Tamil Nadu	2018	October	10	Q4	
4	OD5	Ridhesh	Food Grains	Organic Staples	Ooty	2018-10-11	South	2355	0.28	918.45	Tamil Nadu	2018	October	10	Q4	

Final Data Validation

Checks missing values again after data preparation.

```
( ) df.isnull().sum()
```

```
Order ID      0
Customer Name 0
Category      0
Sub Category   0
City          0
Order Date    0
Region        0
Sales         0
Discount      0
Profit        0
State         0
Order Year    0
Order Month   0
Month Number  0
Order Quarter 0
Profit Margin 0
dtype: int64
```

Checks duplicate rows again after data preparation.

```
( ) df.duplicated().sum()
```

```
np.int64(0)
```

Checks the final data types after cleaning.

Checks duplicate rows again after data preparation.

```
( ) df.duplicated().sum()
```

```
np.int64(0)
```

Checks the final data types after cleaning.

```
( ) df.dtypes
```

```
Order ID      object
Customer Name object
Category      object
Sub Category   object
City          object
Order Date    datetime64[ns]
Region        object
Sales         int64
Discount      float64
Profit        float64
State         object
Order Year    int32
Order Month   object
Month Number  int32
Order Quarter object
Profit Margin float64
dtype: object
```

Checks the final number of rows and columns.

```
( ) df.shape
```

```
(9994, 16)
```

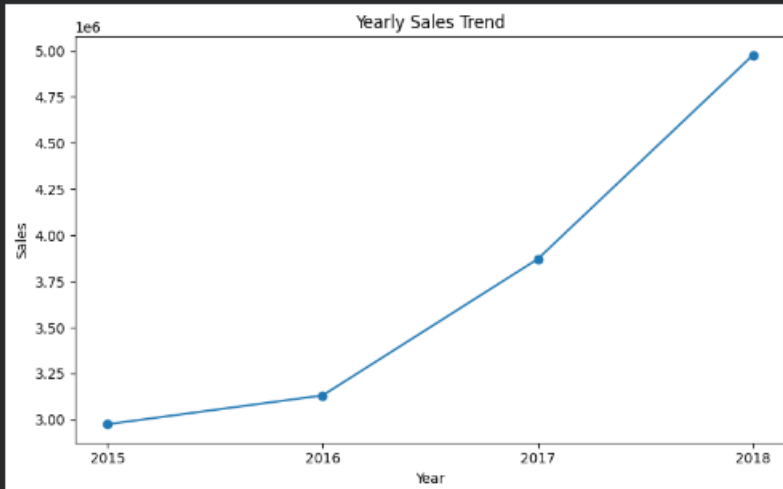
Saves the cleaned dataset for Power BI and further analysis.

```
( ) df.to_csv("Supermart_Grocery_Cleaned.csv", index=False)
```

Shows the yearly Sales trend.

```
yearly_sales = df.groupby("Order Year")["Sales"].sum()

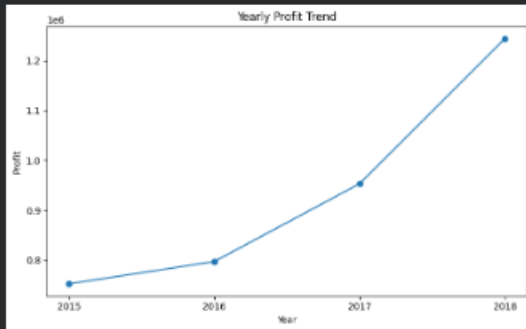
plt.figure(figsize=(8, 5))
yearly_sales.plot(marker="o")
plt.title("Yearly Sales Trend")
plt.xlabel("Year")
plt.ylabel("Sales")
plt.xticks(yearly_sales.index)
plt.tight_layout()
plt.show()
```



Shows the yearly Profit trend.

```
yearly_profit = df.groupby("Order Year")["Profit"].sum()

plt.figure(figsize=(8, 5))
yearly_profit.plot(markers="o")
plt.title("Yearly Profit Trend")
plt.xlabel("Year")
plt.ylabel("Profit")
plt.xticks(yearly_profit.index)
plt.tight_layout()
plt.show()
```



Shows total Sales for each month.

```
month_order = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
]
monthly_sales = df.groupby("Order Month")["Sales"].sum().reindex(month_order)

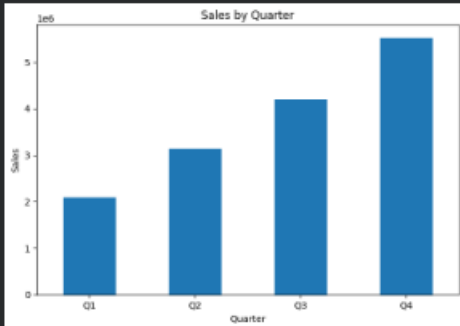
plt.figure(figsize=(12, 5))
monthly_sales.plot(kind="bar")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Shows total sales for each quarter.

```
quarterly_sales = df.groupby("Order Quarter")["Sales"].sum().reindex(["Q1", "Q2", "Q3", "Q4"])

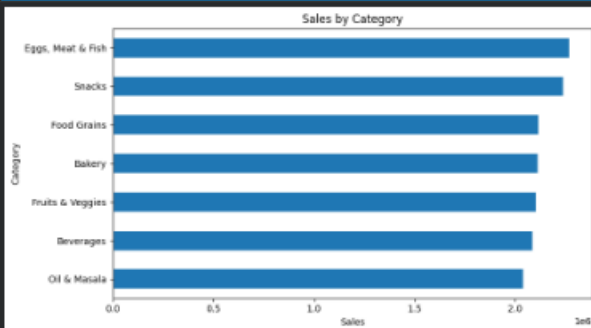
plt.figure(figsize=(7, 5))
quarterly_sales.plot(kind="bar")
plt.title("Sales by Quarter")
plt.xlabel("Quarter")
plt.ylabel("Sales")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Compares Sales across product categories.

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values()

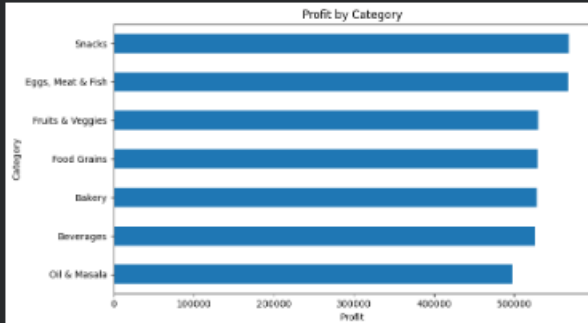
plt.figure(figsize=(9, 5))
category_sales.plot(kind="barh")
plt.title("Sales by Category")
plt.xlabel("Sales")
plt.ylabel("Category")
plt.tight_layout()
plt.show()
```



Compares Profit across product categories.

```
1.1 category_profit = df.groupby("Category")["Profit"].sum().sort_values()

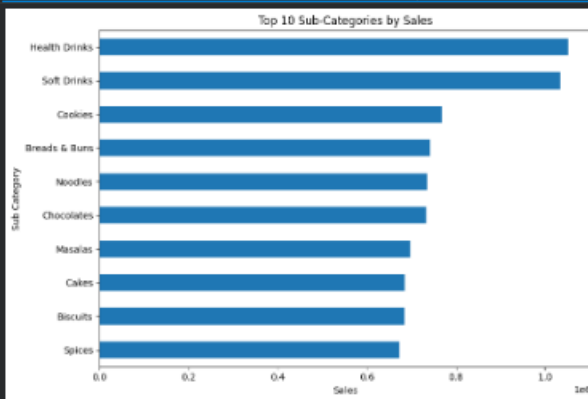
plt.figure(figsize=(8, 5))
category_profit.plot(kind="barh")
plt.title("Profit by Category")
plt.xlabel("Profit")
plt.ylabel("Category")
plt.tight_layout()
plt.show()
```



Shows the top 10 sub-categories by Sales.

```
1.1 top_subcategory_sales = df.groupby("Sub Category")["Sales"].sum().nlargest(10).sort_values()

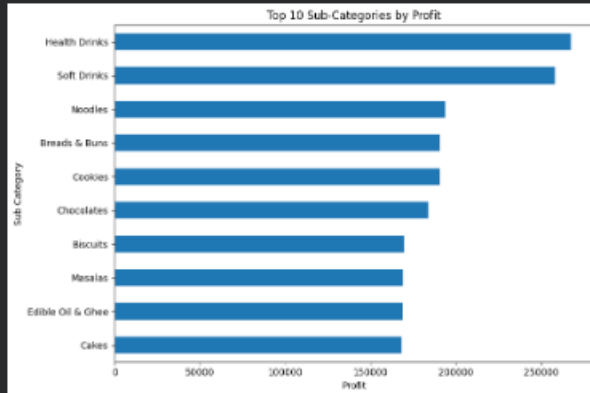
plt.figure(figsize=(8, 6))
top_subcategory_sales.plot(kind="barh")
plt.title("Top 10 Sub-Categories by Sales")
plt.xlabel("Sales")
plt.ylabel("Sub Category")
plt.tight_layout()
plt.show()
```



Shows the top 10 sub-categories by Profit.

```
1 top_subcategory_profit = df.groupby("Sub Category")["Profit"].sum().nlargest(10).sort_values()

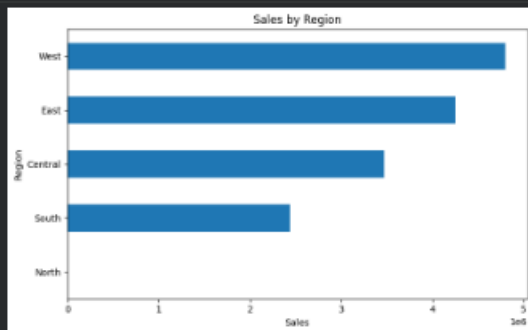
plt.figure(figsize=(9, 6))
top_subcategory_profit.plot(kind="bar")
plt.title("Top 10 Sub-Categories by Profit")
plt.xlabel("Profit")
plt.ylabel("Sub Category")
plt.tight_layout()
plt.show()
```



Compares Sales across regions.

```
1 region_sales = df.groupby("Region")["Sales"].sum().sort_values()

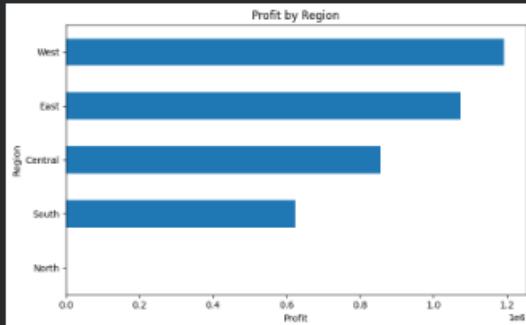
plt.figure(figsize=(8, 5))
region_sales.plot(kind="bar")
plt.title("Sales by Region")
plt.xlabel("Sales")
plt.ylabel("Region")
plt.tight_layout()
plt.show()
```



Compares Profit across regions.

```
region_profit = df.groupby("Region")["Profit"].sum().sort_values()

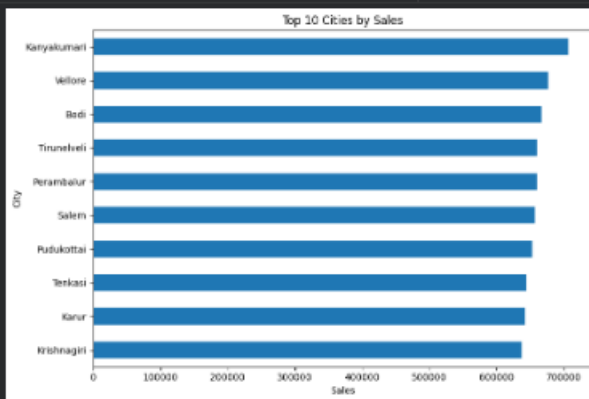
plt.figure(figsize=(8, 5))
region_profit.plot(kind="barh")
plt.title("Profit by Region")
plt.xlabel("Profit")
plt.ylabel("Region")
plt.tight_layout()
plt.show()
```



Shows the top 10 cities by Sales.

```
top_city_sales = df.groupby("City")["Sales"].sum().nlargest(10).sort_values()

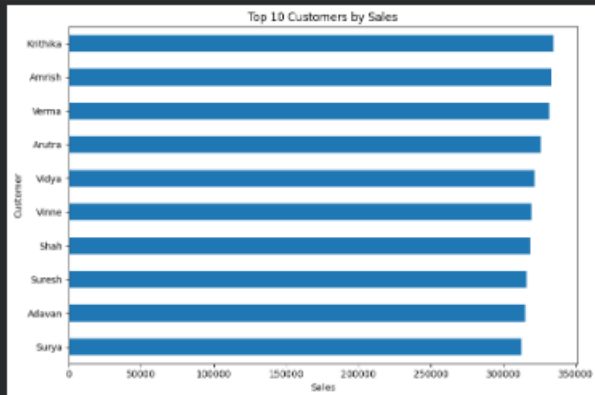
plt.figure(figsize=(9, 6))
top_city_sales.plot(kind="barh")
plt.title("Top 10 Cities by Sales")
plt.xlabel("Sales")
plt.ylabel("City")
plt.tight_layout()
plt.show()
```



Shows the top 10 customers by Sales.

```
top_customer_sales = df.groupby("Customer Name")["Sales"].sum().nlargest(10).sort_values()

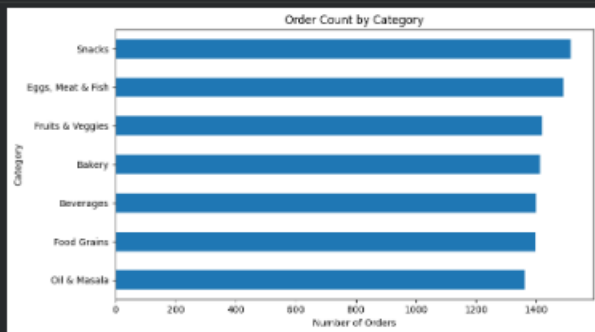
plt.figure(figsize=(9, 6))
top_customer_sales.plot(kind="barh")
plt.title("Top 10 Customers by Sales")
plt.xlabel("Sales")
plt.ylabel("Customer")
plt.tight_layout()
plt.show()
```



Shows the number of orders in each category.

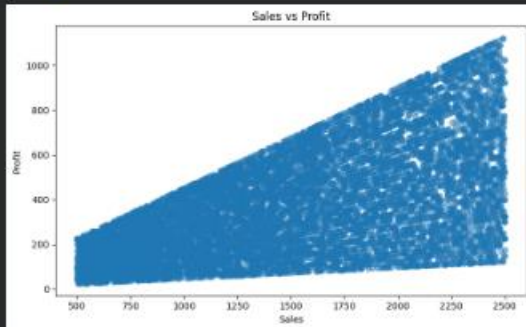
```
category_orders = df.groupby("Category")["Order ID"].count().sort_values()

plt.figure(figsize=(9, 5))
category_orders.plot(kind="barh")
plt.title("Order Count by Category")
plt.xlabel("Number of Orders")
plt.ylabel("Category")
plt.tight_layout()
plt.show()
```



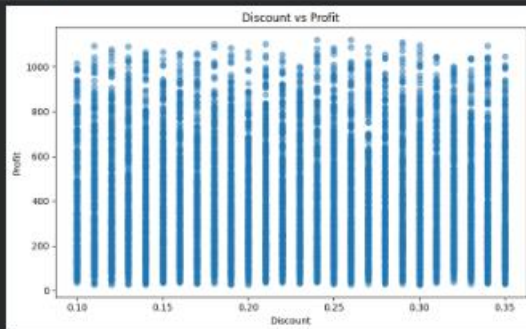
Shows the relationship between Sales and Profit.

```
plt.figure(figsize=(8, 5))
plt.scatter(df['Sales'], df['Profit'], alpha=0.5)
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```



Shows the relationship between Discount and Profit.

```
plt.figure(figsize=(8, 5))
plt.scatter(df['Discount'], df['Profit'], alpha=0.5)
plt.title("Discount vs Profit")
plt.xlabel("Discount")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```



Part 3 – KPI Development

Calculates Total Sales.

```
total_sales = df["Sales"].sum()
print("Total Sales:", total_sales)

Total Sales: 14959982
```

Calculates Total Profit.

```
total_profit = df["Profit"].sum()
print("Total Profit:", round(total_profit, 2))

Total Profit: 3749121.2
```

Calculates the total number of unique orders.

```
total_orders = df["Order ID"].nunique()
print("Total Orders:", total_orders)

Total Orders: 9994
```

Calculates Average Order Value.

```
average_order_value = total_sales / total_orders
print("Average Order Value:", round(average_order_value, 2))

Average Order Value: 1496.6
```

Calculates the overall Profit Margin.

```
overall_profit_margin = (total_profit / total_sales) * 100
print("Profit Margin (%):", round(overall_profit_margin, 2))

Profit Margin (%): 25.05
```

Calculates the total number of customers using unique customer names.

```
customer_count = df["Customer Name"].nunique()
print("Customer Count:", customer_count)

Customer Count: 99
```

Calculates Sales Growth from 2017 to 2018.

```
sales_by_year = df.groupby("Order Year")["Sales"].sum()
sales_growth = ((sales_by_year.loc[2018] - sales_by_year.loc[2017]) / sales_by_year.loc[2017]) * 100
print("2018 Sales Growth (%):", round(sales_growth, 2))

2018 Sales Growth (%): 28.55
```

Model Training:

Future Sales Prediction

Importing the libraries for sales forecasting and model evaluation.

```
[4] ✓ Os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

Loads the already cleaned dataset.

```
[5] ✓ Os
df = pd.read_csv("Supermart_Grocery_Cleaned.csv")
```

Converts Order Date into datetime format because CSV files load dates as text.

```
[6] ✓ Os
df["Order Date"] = pd.to_datetime(df["Order Date"])
```

Creates monthly total sales for time-series forecasting.

```
[7] ✓ Os
monthly_sales = df.set_index("Order Date").resample("MS")["Sales"].sum()
```

Displays the monthly sales data used by the forecasting model.

[8]
✓ Os

monthly_sales



Sales	
Order Date	
2015-01-01	122497
2015-02-01	66030
2015-03-01	247156
2015-04-01	203258
2015-05-01	164263
2015-06-01	206064
2015-07-01	220986
2015-08-01	230161
2015-09-01	382200
2015-10-01	241726
2015-11-01	480979
2015-12-01	410279
2016-01-01	89009
2016-02-01	90823
2016-03-01	212164
2016-04-01	237272
2016-05-01	222722
2016-06-01	205388
2016-07-01	215776
2016-08-01	229543
2016-09-01	429658
2016-10-01	249639
2016-11-01	477720
2016-12-01	472245

Uses the last 12 months as test data and the earlier months as training data.

```
[9]  
✓ Os      train = monthly_sales.iloc[:-12]  
          test = monthly_sales.iloc[-12:]
```

Creates a Holt-Winters forecasting model with trend and yearly seasonality.

```
[10]  
✓ Os      model = ExponentialSmoothing(  
          train,  
          trend="add",  
          seasonal="add",  
          seasonal_periods=12  
      )
```

Trains the forecasting model using the training data.

```
[11]  
✓ Os      model_fit = model.fit()
```

Predicts sales for the 12-month test period.

```
[12]  
✓ Os      predictions = model_fit.forecast(12)
```

Calculates Mean Absolute Error to measure the average prediction error.

```
[13]  
✓ Os      mae = mean_absolute_error(test, predictions)  
          print("MAE:", round(mae, 2))
```



MAE: 44972.95

Calculates Root Mean Squared Error to give more weight to larger prediction errors.

```
[14]  
✓ Os      rmse = mean_squared_error(test, predictions) ** 0.5  
          print("RMSE:", round(rmse, 2))
```



RMSE: 58276.6

Calculates Root Mean Squared Error to give more weight to larger prediction errors.

```
14]  rmse = mean_squared_error(test, predictions) ** 0.5  
✓ Os print("RMSE:", round(rmse, 2))
```

... RMSE: 58276.6

Calculates Mean Absolute Percentage Error to measure prediction error as a percentage.

```
15]  mape = np.mean(np.abs((test.values - predictions.values) / test.values)) * 100  
✓ Os print("MAPE:", round(mape, 2), "%")
```

MAPE: 10.8 %

Creates a table to compare actual and predicted sales.

```
16]  comparison = pd.DataFrame({  
✓ Os     "Actual Sales": test.values,  
     "Predicted Sales": predictions.values  
}, index=test.index)
```

comparison

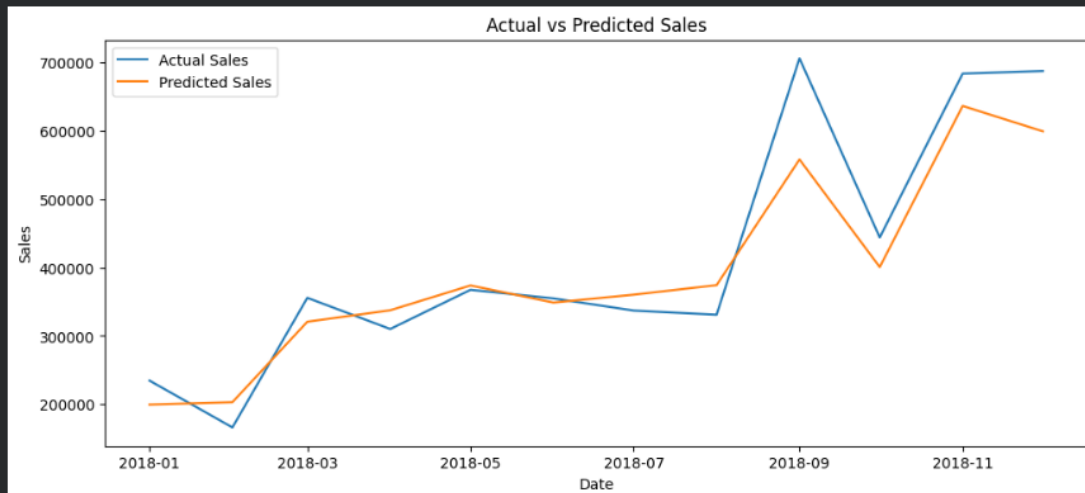
 

	Actual Sales	Predicted Sales
Order Date		
2018-01-01	234739	199700.382494
2018-02-01	166267	203207.518754
2018-03-01	355704	320768.562395
2018-04-01	310150	337528.029965
2018-05-01	367411	373810.625020
2018-06-01	354902	348773.905238
2018-07-01	337092	360326.054336
2018-08-01	331014	374232.772199
2018-09-01	705680	557943.554255

Compares actual sales with predicted sales visually.

[17]
✓ Os

```
plt.figure(figsize=(12, 5))
plt.plot(test.index, test.values, label="Actual Sales")
plt.plot(predictions.index, predictions.values, label="Predicted Sales")
plt.title("Actual vs Predicted Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
```



```
Creates the final forecasting model using all available monthly sales data.

[18]
✓ 0s
final_model = ExponentialSmoothing(
    monthly_sales,
    trend="add",
    seasonal="add",
    seasonal_periods=12
)

Trains the final model using the complete monthly sales history.

[19]
✓ 0s
final_model_fit = final_model.fit()

[23]
✓ 0s
final_model_fit.save("holt_winters_sales_model.pkl")

Forecasts sales for the next six months.

[20]
✓ 0s
future_sales = final_model_fit.forecast(6)

Displays the predicted sales for the next six months.

[21]
✓ 0s
future_forecast = pd.DataFrame({
    "Forecasted Sales": future_sales.round(2)
})

future_forecast

▼
Forecasted Sales
2019-01-01 340619.72
2019-02-01 326120.93
2019-03-01 476022.36
2019-04-01 475799.18
2019-05-01 519586.45
2019-06-01 508138.18
```

Conclusion:

This project transformed the Supermart Grocery Sales data into a complete business analytics solution using Python and Power BI. The analysis identified important sales, product, customer, regional, and profitability patterns. As a bonus, a Holt-Winters forecasting model was developed and evaluated with a MAPE of approximately 10.8% to forecast future monthly sales.